

Design Specification and Implementation Guide for Enhanced School System

CS Capstone Project

Spring 2025

Overview

This document outlines the structure of the School System software project and describes the enhancements that must be implemented. Students will begin with a working codebase that handles member registration, class enrollment, and grade management, and expand it with the following improvements:

- **File Synchronization:** All changes to global lists (Members, StudentCourse, StudentQuizGrades, StudentTestGrades, StudentAssignmentGrades) must immediately be written to their respective CSV files to ensure persistent data integrity.
- **Grade Validation:** Quiz and test grades must be integers in the range 0–100. Assignment grades must be floats between 0 and 1. Entries outside of these ranges must be rejected.
- **Grade Computation:** Each role should be able to compute quiz averages, test averages, and assignment sums. Final grades are calculated using the formula:

$$\text{FinalGrade} = 0.4 \times \text{QuizAvg} + 0.5 \times \text{TestAvg} + \text{AssignmentSum}$$

- **Role Constraints:** Teachers may calculate individual or class-level grade summaries. Students can view only their own statistics.
- **Data Modification Feedback:** Any method that modifies data must print a success/failure message and return a boolean result.
- **Student Assignment Rules:** When assigning a student to a teacher, ensure they are not already assigned. When reassigning, remove from the original teacher before adding to a new one.
- **Administrative Tools:** Add functions to reassign a student, remove a student from a course, and compute transcript-like summaries.
- **Additional Features:** Course-specific averages and final grades should be computed per teacher class.

Class and Method Structure

Python Function Signatures and Specifications

```
# === Design Specification: School System ===
# Global Structures
Members = []
StudentCourse = []
StudentQuizGrades = []
StudentTestGrades = []
StudentAssignmentGrades = []

# --- Utility Functions ---
def hash_password(pw):
    # Fully Implemented
    pass

def member_exists(schoolID):
    # Fully Implemented
    pass

def add_member_to_file(name, schoolID, accountName, password, memberType):
    # Fully Implemented
    pass

def load_members():
    # Fully Implemented
    pass

def load_student_course():
    # Fully Implemented
    pass

def load_grades(filename, target_list):
    # Fully Implemented
    pass

# --- Class Definitions ---

class Member:
    def __init__(self, schoolID, password, name="default", accountName="default", memberType="default"):
        # Fully Implemented
        pass

    def change_password(self, old_password, new_password):
        # Fully Implemented
        pass

class Student(Member):
    def __init__(self, schoolID, password):
        # Fully Implemented
```

```
        pass

    def view_grades(self):
        # Fully Implemented
        pass

    def compute_quiz_avg(self):
        # New: Compute average of quiz grades for this student (0-100 scale)
        pass

    def compute_test_avg(self):
        # New: Compute average of test grades for this student (0-100 scale)
        pass

    def compute_assignment_sum(self):
        # New: Compute total of assignment grades (0-1 values). Return 0 if
            none.
        pass

    def compute_final_grade(self):
        # New: Compute final grade as 0.4*QuizAvg + 0.5*TestAvg +
            AssignmentSum
        pass

class Teacher(Member):
    def __init__(self, schoolID, password):
        # Fully Implemented
        pass

    def add_grade(self, studentID, grade, gradeType):
        # Altered: Validate grade values and persist to CSV after change
        pass

    def compute_student_quiz_avg(self, studentID):
        # New: Compute quiz average for a particular student
        pass

    def compute_student_test_avg(self, studentID):
        # New: Compute test average for a particular student
        pass

    def compute_student_assignment_sum(self, studentID):
        # New: Compute assignment sum for a particular student
        pass

    def compute_student_final_grade(self, studentID):
        # New: Compute final grade for a particular student
        pass

    def compute_class_quiz_avg(self):
        # New: Compute average quiz grade across all students for this
            teacher
```

```
        pass

    def compute_class_test_avg(self):
        # New: Compute average test grade across all students for this
        teacher
        pass

    def compute_class_assignment_sum(self):
        # New: Compute total assignment sum across all students for this
        teacher
        pass

    def compute_class_final_grades(self):
        # New: Compute all final grades for students assigned to this
        teacher
        pass

class Administrator(Member):
    def __init__(self, schoolID, password):
        # Fully Implemented
        pass

    def assign_student_to_teacher(self, teacherID, studentID):
        # Altered: Ensure student is not already assigned to any teacher
        pass

    def remove_student_from_teacher(self, teacherID, studentID):
        # New: Remove student from given teacher and update CSV
        pass

    def reassign_student_to_new_teacher(self, oldTeacherID, newTeacherID,
        studentID):
        # New: Remove student from old teacher and assign to new teacher
        pass

    def display_members(self, memberType):
        # Fully Implemented
        pass

class SuperAdmin(Administrator, Teacher):
    def __init__(self, schoolID, password):
        # Fully Implemented
        pass

    def _create_member(self, name, schoolID, accountName, password,
        memberType):
        # Fully Implemented
        pass

    def create_teacher(self, name, schoolID, accountName, password):
        # Fully Implemented
        pass
```

```
def create_student(self, name, schoolID, accountName, password):
    # Fully Implemented
    pass

def create_admin(self, name, schoolID, accountName, password):
    # Fully Implemented
    pass

def create_superadmin(self, name, schoolID, accountName, password):
    # Fully Implemented
    pass

def reset_password(self, schoolID, new_password):
    # Fully Implemented
    pass

# --- Main Entry Point ---
def main():
    # Fully Implemented
    pass

if __name__ == "__main__":
    main()
```